
Data Structures & Algorithms Project Proposal

for

Cursed Tomorrow

Version 1.0 approved

Prepared by Ahmad 221775

BS Software Engineering, 3rd Semester

Table of Contents

Table of Content	1
1-Introduction.....	3
2- Problem statements.....	3
3- Data Structures and Algorithm's concept	4
4- Solution	4
4.1. Stack	4
4.2. Queue	5
4.3. Dequeue	5
5 -Data Flow	5
1. Game Steps:	6
2. Activity Sequence:	7
3. Activity Diagram:	8
4. Class Diagram:.....	10
6-Conclusion.....	12

1-Introduction

In this document, we will be introducing "Cursed Tomorrow," developed using "Raylib" library; a magnificent 2D platformer that transcends the boundaries of traditional gaming. This game has been crafted using C++ and principles of Data Structures and Algorithm's (DSA). The game uses special graphics designed especially for "Cursed Tomorrow" game. The game also uses efficient methods & algorithms to provide the user interface and game play as better as possible experience to the user.

Storyline:

In "Cursed Tomorrow," failure is not the end; it is a stepping stone on the path to greatness. Rise from the ashes of defeat and let the wisdom gained from each setback propel you towards an extraordinary triumph. As you embark on this journey, prepare to be thrilled at each level .Uncover the hidden meanings behind the instructions, for they hold the key to unlocking the profound secrets of the sprite's destiny and the mysteries that shroud the very essence of "Cursed Tomorrow."

2- Problem statements

In development of "Cursed tomorrow" using previous concepts (object-oriented programming), we ran into following problems: High usage of dynamic memory allocation, tightly bound classes make it difficult to modify or update the code, changes in base classes effects the derived class too, memory overhead due to additional information in classes and objects.

3- Data Structures and Algorithm's concept

- Stack

Stack is used in the code as a screen changer. It works like as the game starts, pushes the main screen in the stack and pops the screen before entering the second screen

- Queue

Queue used in end screen animation when user completes all levels and animations start. Queue provides a simple sequence of animation screen making it easy to add and remove

- Dequeue

Dequeue switches level from one to next. Dequeue updated after the completion of each level

- Iterators

Iterators are used to access the functions without copying the exact data, thus by using only its reference. The game performance enhances and memory consumption decreases

4- Solution

Previous provided data structures and algorithm's concept's help us in the "Cursed Tomorrow". The explanation of process is provided below

4.1. Stack

Stack follows the LIFO order (last in first out). Each screen of the menu displays a state of the game, such as levels, game, credentials, etc. Stack works according to the state of the game. Upon starting a game, the main menu screen is pushed onto the stack. As

after pressing levels, the menu screen popped out and level screen pushed onto the stack and same after the completion of the previous level popped and next level screen pushed onto the screen. The benefit of a process is that it can be easily tracked or navigated by users.

4.2. Queue

Queue follows the order of FIFO (first in, first out). The end game animations work in an organized manner and execute a proper sequence of tasks. The queue is checked to see if it's empty, and if it's not, pushes tasks into it. The process is executed by pushing the tasks into the queue. Pop the tasks after completion. The process is repeated by the system until all tasks are finished and the queue is empty.

4.3. Dequeue

Dequeue also known as “double ended queue” works on the concept of insertion and removal on both sides. Dequeue changing the level from one to next. At the start of gameplay, the current level or level number one is indicated by the front of the queue. As the level completes, it removes the current completed level from the front of the queue and updates the queue. Remove the level if it fails at any level and connect the dequeue to the fail screen.

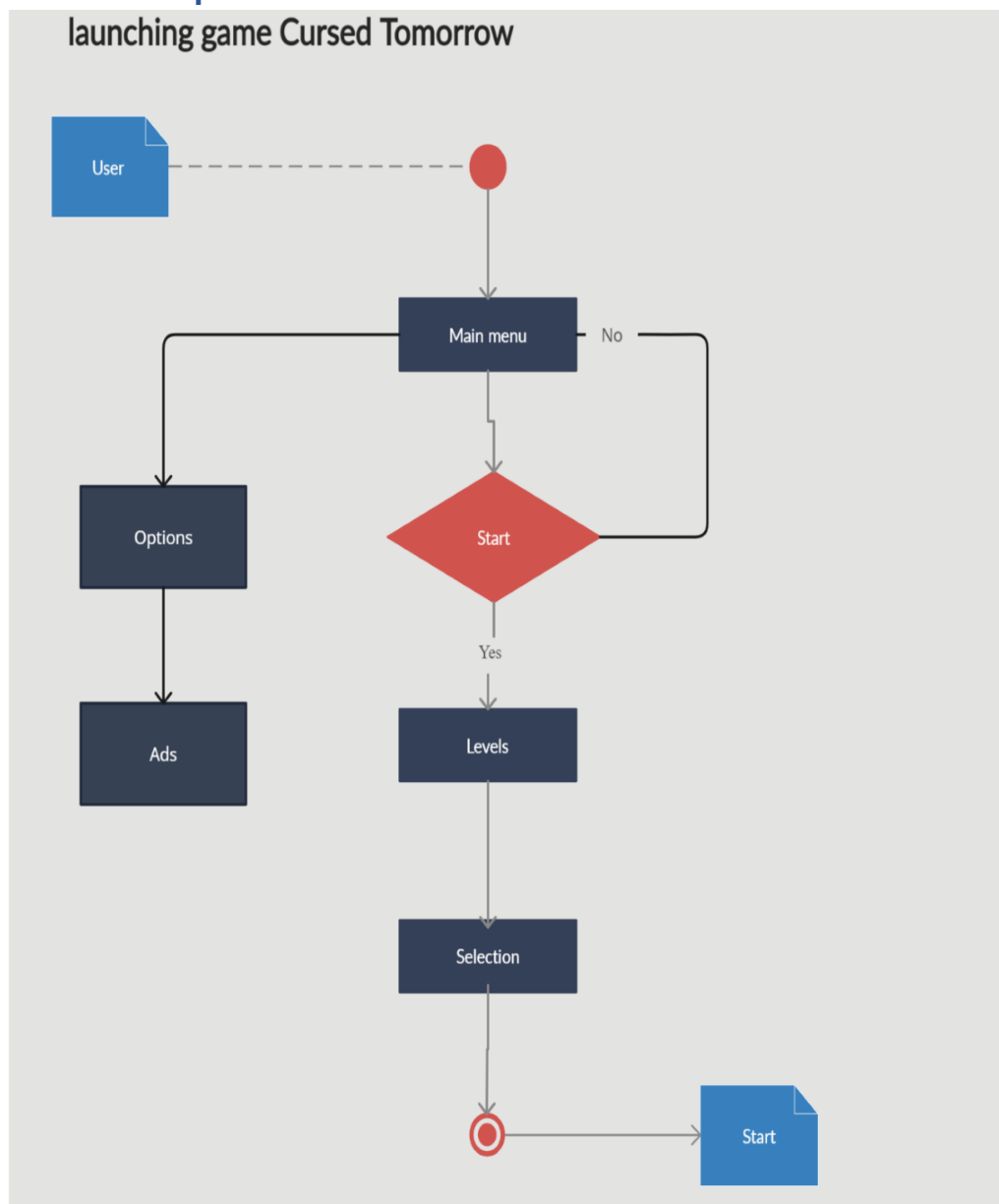
4.4. Iterator

Iterator is a pointer that is used to represent the position of an object. Iterators update and manage the position of an object. As the game starts, the iterator starts checking the position with the help of the position properties of an object, checking the location or position, continues updating with the object movement so that object work properly as the game progress

5 -Data Flow

The following data flow is a behavioral diagram that depicts the movement of activities and actions inside a system or process. It presents the sequential and parallel activities, decision points, and control flow diagrams visually.

1. Game Steps:



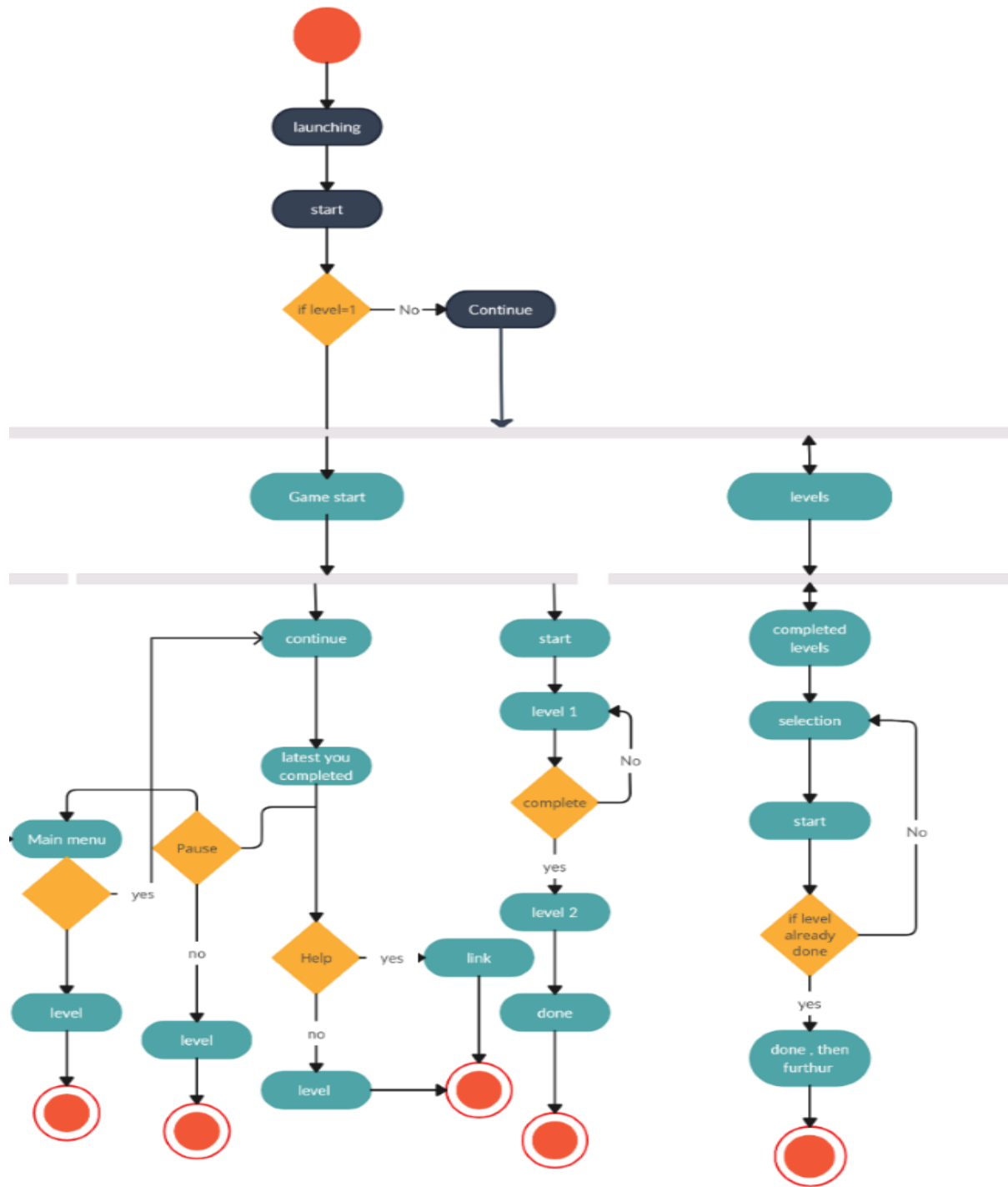
2. Activity Sequence:

After the start of games, these are the following few steps:

- **Start**
- **Level 1**
- **Continue**
 - **If level not equal to 1 then start from latest**
 - **Latest level**
- **Options**
 - **Help**
 - **Pause**
 - **Resume**
 - **Sound**
 - **Main menu**

3. Activity Diagram:

The "Cursed Tomorrow" game shall begin and show the main menu at the beginning. The level selection screen shall appear when the user decides to play. The next step is choosing a level to begin with. The game should show the level screen when the level begins, and the player then continues to play the level. The game ought to verify level completion after completion and show level progress. The player can carry on playing after completing the level and unlocking the following one. Keep playing and progressing through the levels, solving each puzzle, or overcoming each challenge until you reach the end of the game or your desired stopping point.



4. Class Diagram:

The class diagram for "Cursed Tomorrow" shows the different classes that are essential for the game's functioning and organization. At the heart of the diagram is the Main class, which acts as the main hub of the game, coordinating and managing all the different parts.

5.4.2 PNGS:

The PNGS class handles the visual elements of the game, like the pictures and graphics, making sure they are loaded and displayed correctly. The Sound class takes care of the game's audio, including the background music and sound effects.

5.4.3 Stages:

The Stages class represents the different levels or stages in the game, defining their unique features and gameplay rules. The Character class represents the main character that players control, specifying its abilities and how it interacts with the game world.

5.4.4 Levels:

The Levels class manages individual game levels, storing their specific details such as layout, objectives, and difficulty. The Level Manager class oversees the progression of levels, ensuring they are loaded and completed correctly.

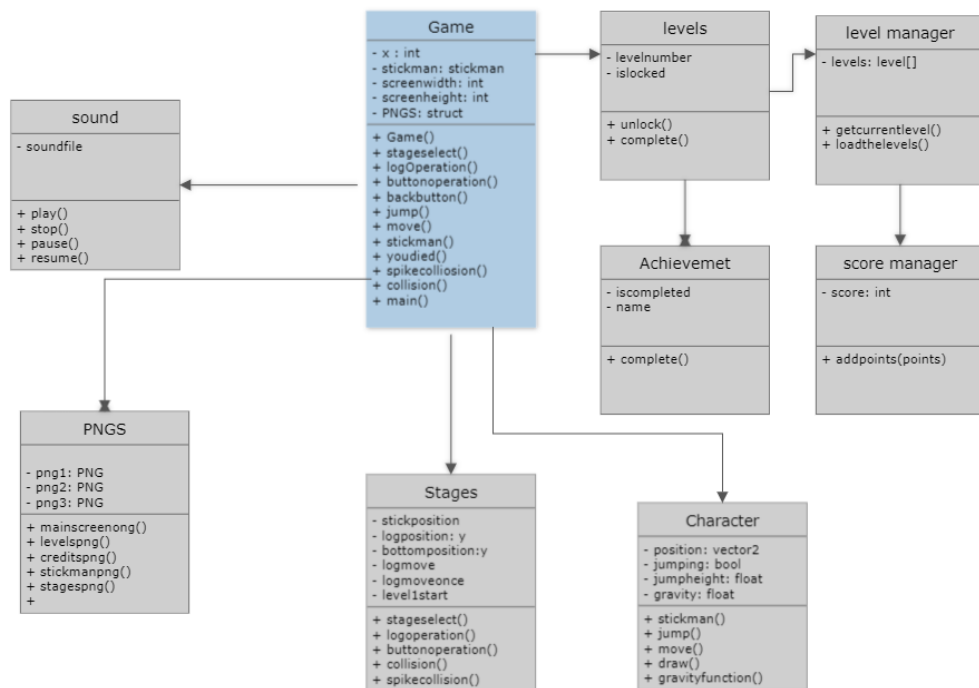
5.4.5 Character:

The Character class represents the player's character or avatar within the game. It encompasses various attributes and functionalities that define the character's appearance, abilities, and progression throughout the game. This class manages aspects such as the character's health, inventory, equipment, and skill set.

Within the Character class, there may be sub-classes or components that further enhance the character's capabilities. For example, there might be a Skill class that handles the character's special abilities or an Inventory class that manages the items the character can collect and use.

The Character class interacts with other classes in the system, such as the Levels class, to determine the character's position and behavior within each level. It may also interact with the Score Manager class to update the player's score based on their performance in completing objectives or defeating enemies.

Overall, the Character class is pivotal in providing the player with agency and control within the game world. It encapsulates the character's attributes, abilities, and progression, allowing players to immerse themselves in the captivating experience of "Cursed Tomorrow."



6-Conclusion

“Cursed Tomorrow” a 2D game developing using the Raylib library, with the help of C++ and DSA concepts. Six different levels narrating a unique story. By addressing the challenges of using OOP concepts, the DSA concept code offers greater efficiency and portability. Stack ensures smooth transformations between the screens, queue handles the end game animations in a proper sequence, dequeue efficiently adopts the levels, and iterators successfully adopt the progress of the game. The data structures and algorithm concepts in code, such as stack, queue, dequeue, and iterators, not only optimize memory usage but also make the structure of the code more efficient.